Representation Learning from Mass Spectrometry Imaging using VAEs

1st Jason Wille
School of Computer Science and Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa
1352200@students.wits.ac.za

***Abstract*—Mass Spectrometry Imaging (MSI) enables the spatial visualisation of molecular distributions within biological tissues by acquiring a full mass spectrum at each spatial pixel. The resulting datasets are extremely high-dimensional and complex, motivating the use of machine-learning techniques for automated feature extraction and interpretation. This work investigates the application of a Variational Autoencoder (VAE) for unsupervised representation learning on MSI spectra. A preprocessing pipeline was implemented to bin and normalise raw spectra, producing a consistent input representation for the model. The VAE was trained to compress each spectrum into a low-dimensional latent space and reconstruct it from this representation, with the goal of uncovering underlying spectral structure. Despite extensive experimentation (including the introduction of a β-VAE formulation to balance reconstruction and regularisation, and modifications to the decoder activation and loss function) the model struggled to learn a discriminative latent representation. Most spectra were reconstructed as a near-identical average profile, indicating limited utilisation of the latent space. These results highlight the challenges of applying probabilistic generative models to MSI data and emphasise the need for architectures and regularisation schemes tailored to the spectral and spatial characteristics of this domain. Nonetheless, the developed framework provides a reproducible foundation for future research into deep generative modelling of molecular imaging data.**

I. Introduction

Mass Spectrometry Imaging is an analytical technique that allows the spatial mapping of molecular species across biological tissue sections.

In recent years, MSI has seen many improvements and, according to Hu and Laskin [1], is an important technique in biological research, clinical studies, drug discovery, and other fields. One of the advances in MSI has been an increase in spatial resolution, which has led to the need for better tools for data visualisation and data analysis [2].

According to Hu and Laskin [1], the number of publications related to AI and MSI had increased exponentially over the period between 2016 and 2021. This is likely due to advances in computational efficiency, improvements in deep learning techniques, and a growing interest in their applications to different fields.

In MSI, mass spectra that contain thousands of mass-to-charge (m/z) readings at each pixel of the image are collected [3]. This leads to high-dimensional data and the need for effective dimensionality reduction techniques that do not lose meaningful data, but that do remove unnecessary noise. These representation learning methods can be referred to as dimensionality reduction or data compression techniques and allow a more compact representation of the original data to be used to perform data analysis. This is desirable as it helps to extract the most useful signals from the data, which in turn allows for more accurate data analysis.

The present work investigates the use of a variational autoencoder to learn an unsupervised, low-dimensional representation of MSI spectra. The approach aims to:

1) Compress high dimensional spectra into a compact latent embedding.
2) Explore the latent structure through clustering and manifold-learning techniques such as t-SNE and UMAP.
3) Interpret clusters in spectral domains to identify meaningful biochemical variations.

By examining how MSI data is organised in the learnt latent space, this work seeks to demonstrate how deep generative models can support exploratory molecular analysis and contribute to more interpretable data-driven insights in biomedical research.

In the remainder of this report, Section II provides the theoretical background of variational autoencoders, as well as some of the computational methods currently used in MSI for data analysis. Section III describes the methodological framework, detailing data preprocessing, model architecture, training procedures, and clustering and visualisation of the learned latent space. Section IV presents the experimental results, including training behaviour, latent-space organisation, and analysis of cluster centroids. Section V concludes with a discussion that recontextualises the findings within the biomedical imaging domain, highlighting the implications of VAE-based representations for future research.

II. Background

A. Deep Learning

The strength that deep learning (DL) provides is based on the deep artificial neural networks (ANNs) that are the foundation of most DL techniques. The term deep neural networks refers to an ANN with many hidden layers. The complexity added to the model by adding more hidden layers allows for complex non-linear relationships to be found in the

data on which the model is being trained. LeCun et al. [4] state that DL methods are representation learning methods that learn different representations built up from the raw input. Each representation adds extra complexity and allows for complex functions to be learned from the data.

One advantage of DL techniques when compared to conventional ML techniques is that DL techniques do not require handcrafted features to be used as input to their models [4]. This means that complex features in the data can automatically be learned by DL techniques. This saves a lot of time and effort; in the past domain expertise would be required in order to design useful features from complex datasets.

1) Artificial Neural Networks: Artificial neural networks are the foundational building blocks for deep learning techniques and are based on a perceptron which, in turn, is based loosely on a biological neuron. The nodes in a neural network take in inputs, multiply them by weights on the different edges and add a bias value. Once this is calculated, an activation function is applied and the output is optionally fed into the next layer.

Equation 1 shows how to calculate the output (also known as the activation) represented by a. The node at which the output is calculated is represented by j. A layer in the neural network is represented by l. A weight connecting node i in layer $l-1$ to node j in layer l is represented by $w_{ij}^{[l]}$. The bias at node j in layer l is represented by $b_j^{[l]}$. The activation at node i in layer $l-1$ is represented by $a_i^{[l-1]}$. Figure 1 shows the general structure of an artificial neural network and Figure 2 shows a visualisation of the calculation of an activation value.

$$a_j^{[l]} = g_j^{[l]}\left(\left(\sum_i a_i^{[l-1]} w_{ij}^{[l]}\right) + b_j^{[l]}\right) \tag{1}$$

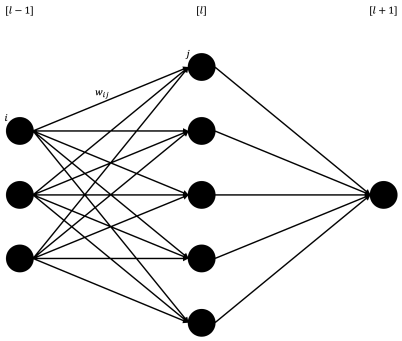


Fig. 1. Structure of a fully-connected ANN with 3 layers

Inputs are fed into an ANN and the calculation shown in Equation 1 is done for the nodes starting from the first layer and continuing until the final layer. This results in an output for the whole ANN. This output, as well as the label associated with the input, can be used to calculate a loss. This loss is then used to calculate gradients for all of the weights and bias values using the partial derivatives of the loss with respect to each parameter. The gradients are used to slowly adjust

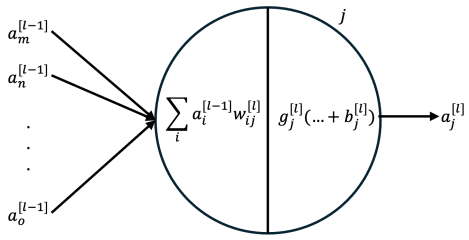


Fig. 2. Activation calculation for node j in layer l with i input activations from layer $l-1$

the parameters and this process is repeated until the loss has reached an acceptable value.

2) Autoencoders: Deep learning methods may be applied to datasets that include explicit labels as well as to those that do not. This distinction gives rise to two broad categories of machine-learning techniques: supervised and unsupervised learning. In supervised learning, a model is trained using labelled data to optimise a performance metric that reflects how accurately the model predicts these labels. In contrast, unsupervised learning seeks to uncover structure or patterns inherent in the data without the use of any ground-truth labels.

One type of unsupervised learning technique, known as autoencoders, are of particular interest as they take data from a higher dimensional space and encode the data into a latent space that is usually a lower dimensionality than the original data.

Autoencoders are a specific class of ANN which do not relate an input to a target but rather attempt to take an input, compress it into a lower dimensional space, and then reproject the lower dimensional value to the original input [5]. Hinton and Salakhutdinov [6] states that the autoencoder is a non-linear generalisation of principal component analysis (PCA). Autoencoders are made up of two components, an encoder, and a decoder. The encoder compresses the data to a dimensionality that can fit through the bottleneck and the decoder takes the data from the bottleneck and tries to reproduce the input fed into the encoder. The error is measured as a reconstruction loss between the reconstructed data that the decoder produced and the input data. The architecture of a general autoencoder is shown in Figure 3.

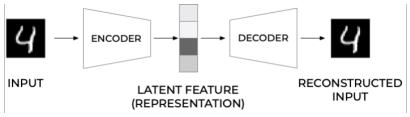


Fig. 3. Architecture of a general autoencoder [7]

The latent space of an autoencoder can be interesting to analyse as it can highlight aspects of the original data that form the most important features. One downside of the latent space of a regular autoencoder is that there are no constraints placed on it. One possible use of an autoencoder is to sample points from the latent space and pass them through the decoder in order to generate data points. This task is not feasible with

a standard autoencoder but one model that does have this generative property is a variational autoencoder.

3) Variational Autoencoder: The variational autoencoder was introduced by Kingma et al. [8] and the general structure of a VAE is shown in Figure 4. The VAE shares the same high-level structure as a standard autoencoder, comprising an encoder and a decoder (denoted by "E" and "D", respectively, in Figure 4). However, it differs fundamentally in that the encoder does not produce a single deterministic latent vector; instead, for each input data point, it outputs a mean μ and standard deviation σ that parameterize a Gaussian distribution in the latent space.

Because the latent representation is obtained by sampling from this Gaussian distribution, the resulting stochasticity would ordinarily make the loss function non-differentiable. However, the reparameterization trick introduced by Kingma et al. [8] enables gradient-based optimisation by expressing the sampling operation in a differentiable form. The reparameterization trick is highlighted in Figure 4 which shows how the variability is obtained from a term, ϵ. This trick allows the loss function to be differentiable and for backpropagation to be applied.

Before the introduction of the reparameterization trick, the latent variable z was obtained by sampling directly from the Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$. Because this sampling operation is inherently stochastic, it breaks the computational graph and prevents gradients from flowing back through μ and σ, making the loss function non-differentiable with respect to the encoder parameters. The reparameterization trick resolves this issue by rewriting the sampling step as $z = \mu + \sigma \odot \epsilon$, where $\epsilon \sim \mathcal{N}(0, 1)$; in this form, the randomness is isolated in the noise term, while μ and σ remain part of a differentiable computation.

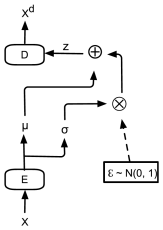


Fig. 4. Structure diagram of a VAE [9]

When training a VAE, there is an interest in different distributions that describe the input data and the latent space and their relationship with one another. The data comes from a distribution $p(x)$, while the distribution of the latent variable z comes from $p(z)$. The distribution $p(z|x)$ allows us to get the latent variable z for a given data point x. These conditional and marginal distributions are linked using Bayes' theorem shown in Equation 2 and the glossary for the names of these distributions is shown in Table I.

$$p(z|x) = \frac{p(x|z)p(z)}{p(x)} \tag{2}$$

TABLE I
THE DIFFERENT PARTS OF BAYES' THEOREM [10]

Symbol	Name
z	Latent variable
$p(z)$	Prior probability
$p(z\|x)$	Posterior probability
x	Evidence or data
$p(x)$	Evidence probability
$p(x\|z)$	Likelihood probability

When training a VAE, an approximation of the posterior distribution, which is denoted as $q_\theta(z|x)$ is learnt. θ represents the distribution parameters and can be thought of as the weights of the encoder portion of the network. The decoder portion of the network and its weights represent the likelihood distribution denoted as $p_\phi(x|z)$. This is shown in Figure 5.

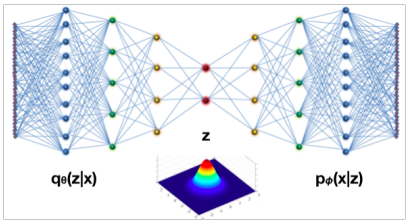


Fig. 5. Distributions learnt by the encoder and decoder of a VAE [10]

The approximation of the posterior distribution, $q_\theta(z|x)$, should be as close to the posterior distribution, $p(z|x)$, as possible. The KL divergence is used to measure the difference between these distributions and is shown in Equation 3. The prior distribution is assumed to be the unit Gaussian distribution.

$$D_{KL}\Big[q_\theta(z|x)\|p(z|x)\Big] = E_{q_\theta(z|x)}\Big[\log(q_\theta(z|x)) - \log(p(z|x))\Big] \tag{3}$$

Equation 3 can be simplified to show that the cost for the VAE can be represented as Equation 4.

$$J_{\text{VAE}} = -D_{KL}\Big[q_\theta(z|x)\|p(z)\Big] + E_{q_\theta(z|x)}\Big[\log p_\phi(x|z)\Big] \tag{4}$$

The first term of the loss function is used to constrain the shape of the posterior approximation. The second term of the loss function is called the reconstruction term [10] because it measures the likelihood of the data reconstructed by the decoder.

4) β-VAE: A widely used extension of the standard VAE framework is the β-VAE, which introduces a scaling factor β to the KL divergence term in the cost function, shown in Equation 5.

$$J_{\beta-\text{VAE}} = -\beta D_{KL}\Big[q_\theta(z|x)\|p(z)\Big] + E_{q_\theta(z|x)}\Big[\log p_\phi(x|z)\Big] \tag{5}$$

The parameter β is typically chosen to be greater than one, thereby increasing the strength of the regularisation term that

encourages the approximate posterior $q_\theta(z|x)$ to align more closely with the unit Gaussian prior $p(z)$. This stronger constraint is often associated with improved disentanglement of the latent factors [11], making the learned representations more interpretable and better aligned with underlying generative factors in the data. However, increasing β also forces the latent distribution to adhere more tightly to the prior, which can reduce the model's capacity to encode fine-grained information and thus generally leads to poorer reconstruction quality.

B. Computational Methods in MSI

1) Mass Spectrometry Imaging: Mass spectrometry imaging is a technique that allows high-quality imaging of biological samples [1]. MSI data is captured by taking mass spectra in different x, y coordinates in order to form a hyperspectral image. The general process followed in an MSI experiment is shown in Figure 6. An experiment starts by collecting the sample that will be used and then preparing it for the MSI instrumentation that will be used to capture the spectra. The next phase of the experiment involves capturing the spectra of the sample at different x, y coordinates and then processing and visualising the raw data. Finally, the data processing section of the experiment is the last phase where different computational techniques can be applied to the data collected.

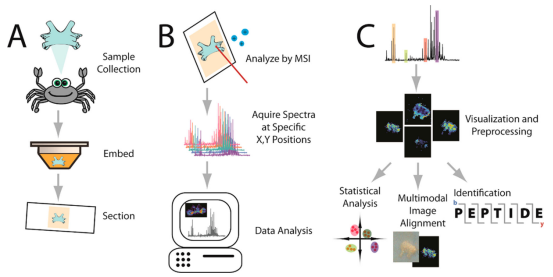


Fig. 6. Procedure for a general MSI experiment highlighting the importance of computational techniques [12]

According to Fonville et al. [13], most biochemical analyses do not take into account spatial distributions, which is vital for biomedical research. This is where the strength of MSI comes into play, as MSI can represent both spatial and spectral information by measuring different mass-to-charge spectra over a spatial grid. MSI data can be interpreted as mass spectra at a specific x, y coordinate (or pixel), or it can be interpreted as an image of a specific ion's intensity over a 2D set of pixels; this is highlighted by the general structure of the MSI data cube shown in Figure 7.

In recent years there have been advances in the spatial resolution and acquisition throughput of MSI [1]; this has led to an increase in the size of a single data point captured in an MSI experiment. This increase in the size of data points has highlighted the need for more efficient methods for data visualisation and analysis.

2) Data Compression: The 3D data cube shown in Figure 7 is often considered for data compression after spectral data processing has been carried out on the data. The spectral data

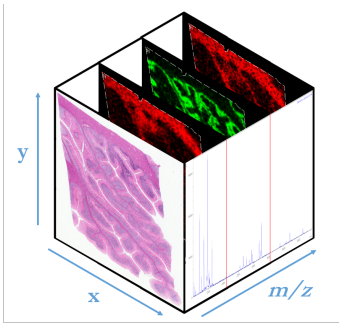


Fig. 7. MSI data cube highlighting the relationship between spatial and spectral information [14]

processing tries to remove some noise from the data by using methods such as peak picking and signal normalisation. Even after this noise has been removed from the data, the data still has high dimensionality. This leads to the need for useful data compression and dimensionality reduction techniques.

According to Hu and Laskin [1], matrix factorisation techniques such as PCA, non-negative matrix factorisation, and multivariate curve resolution-alternating least squares have been used extensively. Methods such as self-organised maps, t-distributed stochastic neighbour embedding (t-SNE), and uniform manifold approximation and project (UMAP) have also gained popularity. Self-supervised learning has also gained popularity in the space of deep learning models as it avoids the need to have labelled data. Thomas et al. [15] were the first to propose an autoencoder architecture to compress MSI spectral data.

One of the benefits found when using neural network based approaches compared to t-SNE and UMAP is that neural network based approaches allow for models to be trained in batches which allows for only portions of the large MSI data to be present in RAM [7].

Another consideration is that data can be compressed in a spectral and a spatial manner. There has been more research into methods of compression for spectral data, but spatial compression has not been well explored. Hu and Laskin [1] predict that self-supervised learning will provide enhanced computational methods for MSI data in the future.

III. Method

A. Data Preprocessing

The original MSI dataset consisted of $160{,}000$ spectra, each corresponding to a single spatial pixel of a 400×400 image. Each spectrum contained intensity measurements recorded at varying ranges of mass-to-charge (m/z) ratios. Across all spectra, the minimum observed m/z ratio was approximately 100, and the maximum was approximately 1500.

To create a standardised input suitable for training the VAE, all spectra were binned along a common m/z axis with a fixed bin width of 0.02 Da. This discretisation resulted in spectra represented by $70{,}000$ intensity values per pixel, each corresponding to a specific m/z interval. The bin centres were

computed as the midpoints between consecutive bin edges, ensuring consistent alignment of spectral features across all pixels.

Before binning, each spectrum was normalised using *total ion current* (TIC) normalisation, in which the intensity at each m/z value is divided by the sum of all intensities in that spectrum:

$$I'_i = \frac{I_i}{\sum_j I_j} \tag{6}$$

This procedure scales each spectrum such that its total intensity equals one, mitigating the influence of pixel-to-pixel variation in overall ion abundance. After TIC normalisation and binning, every spectrum shared a uniform m/z grid and comparable intensity scale, providing a consistent numerical representation for model training.

B. Model Architecture and Training

A variational autoencoder was implemented to learn a compact latent representation of the binned and normalised MSI spectra. Each spectrum, represented as a one-dimensional vector of 70,000 intensity values, served as an input sample to the network. The model was implemented in `PyTorch` and trained on a GPU to accelerate computation.

1) Architecture: The VAE consisted of two fully connected components: an encoder and a decoder. The encoder compressed the high-dimensional input spectrum into a latent distribution parametrised by a mean vector μ and a log-variance vector $\log \sigma^2$. The encoder comprised a sequence of linear layers with progressively decreasing dimensionality, interleaved with rectified linear unit (ReLU) activation functions. The decoder mirrored this structure, expanding the latent representation back to the original input dimensionality to reconstruct the normalised spectrum. A final sigmoid activation ensured that reconstructed intensities remained in the range $[0, 1]$. The latent space dimensionality was set to $d_z = 200$, providing a balance between compression and expressivity.

Batch normalisation was also applied between the different dense layers of the encoder and decoder.

The number of neurons in the different layers of the network is shown in Figure 8 and were as follows: 70000, 2048, 512, 200, 512, 2048, and 70000.

2) Loss function: The network was trained to minimise the standard VAE loss, which combines a reconstruction loss and a regularisation term. The reconstruction loss was computed using mean squared error (MSE) between the input spectrum x and its reconstruction $\hat{x}$, while the Kullback–Leibler (KL) divergence penalised deviations of the latent posterior distribution $q_\theta(z|x)$ from the unit Gaussian prior $p(z)$.

3) Training Procedure: The model was trained for 20 epochs using the AdamW optimiser with a learning rate of 1×10^{-3}, a batch size of 64, and a weight decay of 1×10^{-2}. The total loss, reconstruction loss, and KL divergence were monitored throughout training to assess convergence and latent-space stability.

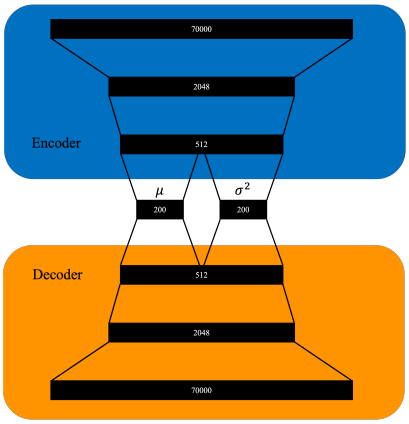


Fig. 8. VAE architecture used to obtain results

After training, the encoder was used to extract the mean latent vector μ for each input spectrum, forming a continuous low-dimensional representation of the MSI dataset suitable for clustering and visualisation.

IV. Results

This section presents the results of training the variational autoencoder on the preprocessed MSI dataset, as well as the subsequent analyses of the learned latent space and its biological interpretation. The results are organised into two parts: training performance, and latent-space organisation.

A. Training Performance

The VAE was trained for 20 epochs and the different plots of the loss values are shown in Figure 9, 10, and 11. The x-axis of the loss plots is the number of batches that were processed. There were 2500 batches of 64 spectra in a single epoch. The plots highlight that the reconstruction loss decreased as training continued and that the KL divergence also stabilised. This seemed promising as it meant that the VAE was learning the structure of the data as well as constraining the distribution of the latent space.

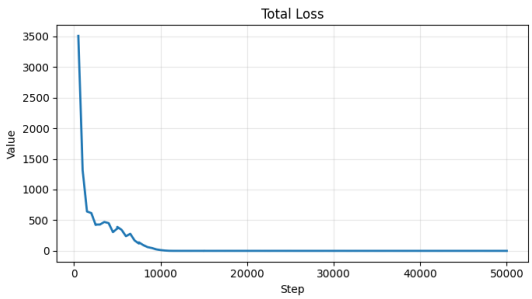


Fig. 9. Total training loss

B. Latent Space Organisation

In order to analyse the structure of the latent space and to search for any meaning of the encoded spectra, k-means clustering was performed on the mean values that were encoded

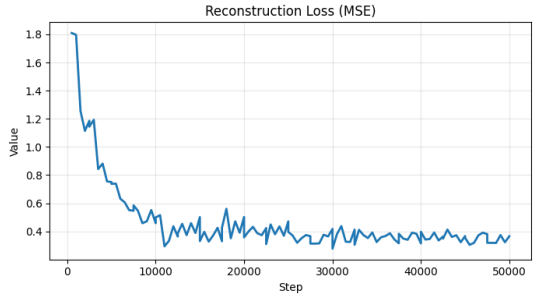


Fig. 10. Reconstruction loss measured as MSE during training

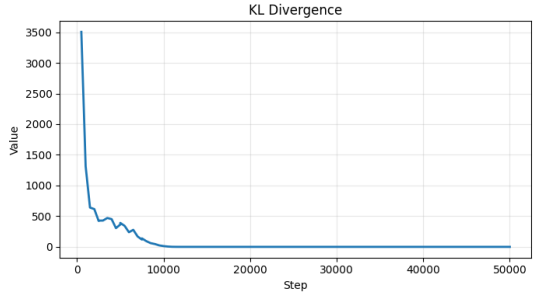


Fig. 11. KL divergence between the estimated posterior distribution and prior distribution measured during training

from the input spectra. The optimal value of k that was chosen was 12, based on the plateau seen in Figure 12.

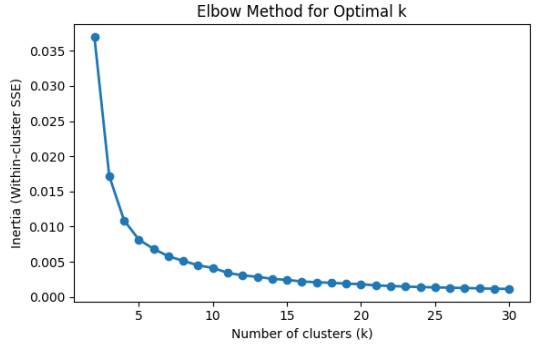


Fig. 12. Elbow plot in order to determine the optimal number of clusters for k-means

After clustering on the latent space was performed, t-SNE and UMAP were performed on the latent space in order to visualise the latent space. The cluster labels that were obtained from the k-means algorithm were also plotted on the t-SNE and UMAP results in order to see if there was any clear separation between the clustered regions in the latent space. The t-SNE and UMAP visualisations are shown in Figure 13, and 14. The distribution of the latent space, as visualised in the t-SNE plot, demonstrates that the latent representations were constrained by the KL divergence term, which enforced alignment with a Gaussian prior.

The different k-means clusters for the $160,000$ spectra were also plotted in different 400×400 pixel images, this is shown

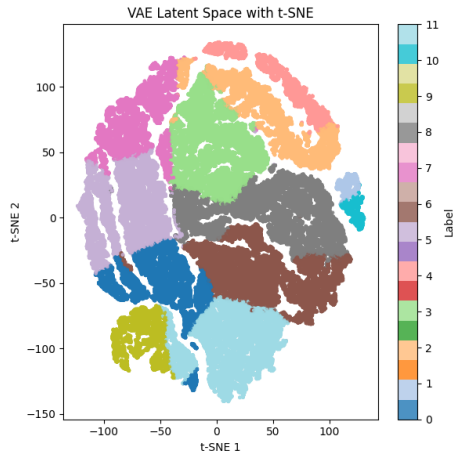


Fig. 13. t-SNE visualisation with cluster labels from k-means

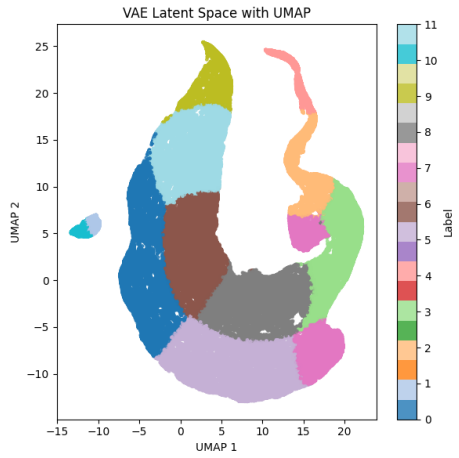


Fig. 14. UMAP visualisation with cluster labels from k-means

in Figure 15.

V. Conclusion and Discussion

This project investigated the use of a variational autoencoder for learning low-dimensional representations of mass spectrometry imaging data. The aim was to compress high-dimensional spectra into a latent space that captures underlying molecular structure, enabling clustering and visual exploration of biochemical variability across tissue sections.

In practice, the model struggled to learn a meaningful representation of the data. During training, it was observed that the VAE tended to reconstruct a similar type of spectrum for nearly all inputs, indicating that the latent variables were not being effectively utilised. This behaviour suggests that the decoder was learning to reproduce an "average" spectrum rather than modelling pixel-specific variation – a form of posterior collapse commonly encountered in VAEs when the decoder dominates the learning process.

To address this, a β-VAE formulation was explored in which the weight of the KL divergence term was adjusted

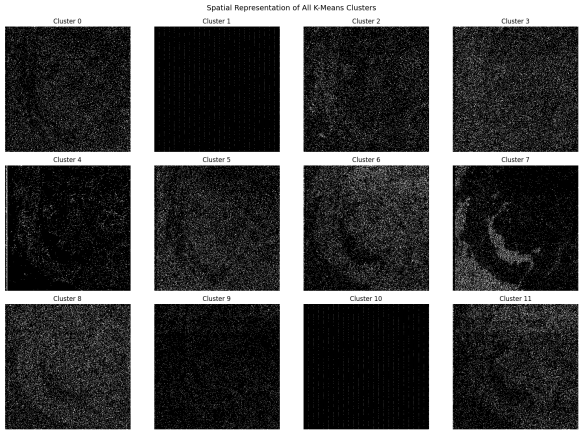


Fig. 15. The 400×400 pixel spatial representations of the k-means clusters

to encourage the encoder to learn a more disentangled latent space. Several values of β were tested in an attempt to balance reconstruction fidelity and latent regularisation, but this did not produce any significant improvement. The model continued to generate highly similar reconstructions across the dataset, suggesting that the latent variables remained underutilised. The mean and maximum spectrum are plotted where the mean intensity value and the maximum intensity values were calculated over all of the spatial coordinates; this can be seen in Figure 16. When investigating the decoded centroids, it was found that they all produced very similar spectra to the mean spectra, which is shown in Figure 17.

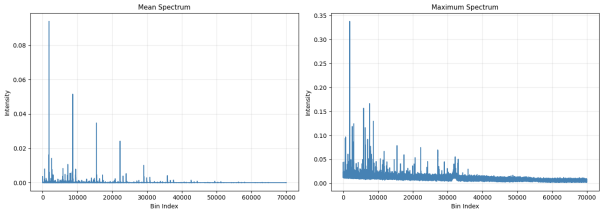


Fig. 16. Average and maximum intensity values across all spatial coordinates

Further modifications were made to the network architecture and loss function. Specifically, the final sigmoid activation in the decoder was removed to allow the model to reconstruct unbounded intensities. Despite these adjustments, the VAE did not converge to a representation that meaningfully separated different spectral or spatial features within the MSI data.

Whilst not all of the latent space proved to be useful, cluster 7, as shown in Figure 15, did manage to capture an important pattern seen in the data which is highlighted by the plot of the original data shown in Figure 18.

Although the results were not as successful as initially anticipated, they highlight several important challenges in applying deep generative models to MSI analysis. The high dimensionality, sparsity, and variability of MSI spectra make learning robust latent representations difficult without architectural or regularisation techniques tailored to the data domain. Future

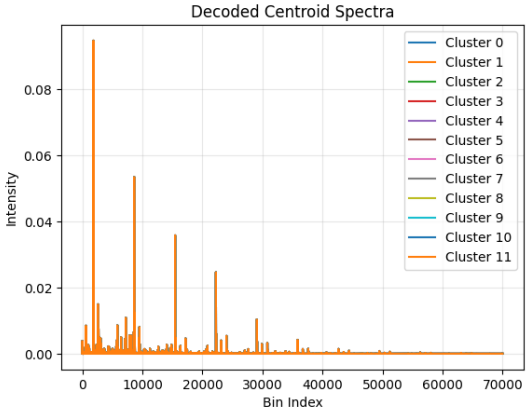


Fig. 17. Decoded latent cluster centroids

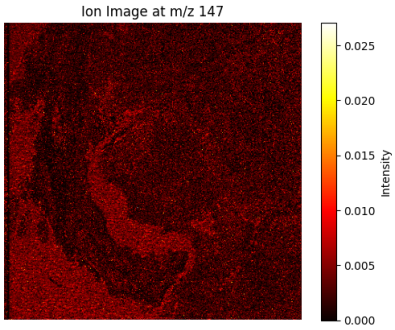


Fig. 18. TIC normalised spatial image at m/z 147

work could explore alternative models such as convolutional, hierarchical, or vector-quantised VAEs [16]. In the biomedical context, this study reinforces the need for domain-informed generative approaches that can integrate molecular and spatial information in a way that remains interpretable to medical practitioners.

VI. AI Statement

I understand that the use of generative AI tools (such as ChatGPT or similar) without explicitly declaring such use constitutes a form of plagiarism and is classified by Wits University as academic misconduct.

I declare that in the course of conducting the research towards my degree or in the preparation of this project report **I did make use of generative AI tools for the following**:

1) ChatGPT: Simplifying complex concepts from research papers.
2) ChatGPT: Generating LaTeX code.
3) ChatGPT: Generating template code used for plotting graphs.
4) ChatGPT: Rewriting short paragraphs in a more formal style.
5) Google Scholar PDF Reader: AI outline of research papers.

If generative AI tools were used as an integral part of the experimental design or in the direct execution of my research,

I confirm that details of this use are clearly outlined in the relevant experimental/methodology chapters of my project report.

References

[1] H. Hu and J. Laskin, "Emerging computational methods in mass spectrometry imaging," *Advanced Science*, vol. 9, no. 34, p. 2203339, 2022.

[2] Y. Zhang and X. Liu, "Machine learning techniques for mass spectrometry imaging data analysis and applications," pp. 519–522, 2018.

[3] P. Bjärterot, A. Nilsson, R. Shariatgorji, T. Vallianatou, I. Kaya, P. Svenningsson, L. Käll, and P. E. Andrén, "Met-id: An open-source software for comprehensive annotation of multiple on-tissue chemical modifications in maldi-msi," *Analytical Chemistry*, vol. 97, no. 16, pp. 9033–9041, 2025, pMID: 40253716. [Online]. Available: https://doi.org/10.1021/acs.analchem.5c00633

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[5] S. Chen and W. Guo, "Auto-encoders in deep learning—a review with new perspectives," *Mathematics*, vol. 11, no. 8, p. 1777, 2023.

[6] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *science*, vol. 313, no. 5786, pp. 504–507, 2006.

[7] U. Michelucci, "An introduction to autoencoders," *arXiv preprint arXiv:2201.03898*, 2022.

[8] D. P. Kingma, M. Welling *et al.*, "Auto-encoding variational bayes," 2013.

[9] P. Li, Y. Pei, and J. Li, "A comprehensive survey on design and application of autoencoder in deep learning," *Applied Soft Computing*, vol. 138, p. 110176, 2023.

[10] S. Odaibo, "Tutorial: Deriving the standard variational autoencoder (vae) loss function," *arXiv preprint arXiv:1907.08956*, 2019.

[11] C. P. Burgess, I. Higgins, A. Pal, L. Matthey, N. Watters, G. Desjardins, and A. Lerchner, "Understanding disentangling in β-VAE," *arXiv preprint arXiv:1804.03599*, 2018.

[12] A. R. Buchberger, K. DeLaney, J. Johnson, and L. Li, "Mass spectrometry imaging: a review of emerging advancements and future insights," *Analytical chemistry*, vol. 90, no. 1, p. 240, 2017.

[13] J. M. Fonville, C. L. Carter, L. Pizarro, R. T. Steven, A. D. Palmer, R. L. Griffiths, P. F. Lalor, J. C. Lindon, J. K. Nicholson, E. Holmes *et al.*, "Hyperspectral visualization of mass spectrometry imaging data," *Analytical chemistry*, vol. 85, no. 3, pp. 1415–1423, 2013.

[14] M. Galli *et al.*, "An easy-to-use software program for the ensemble pixel-by-pixel classification of maldi-msi datasets," 2018.

[15] S. A. Thomas, A. M. Race, R. T. Steven, I. S. Gilmore, and J. Bunch, "Dimensionality reduction of mass spectrometry imaging data using autoencoders," in *2016 IEEE symposium series on computational intelligence (SSCI)*. IEEE, 2016, pp. 1–7.

[16] R. Wei, C. Garcia, A. El-Sayed, V. Peterson, and A. Mahmood, "Variations in variational autoencoders-a comparative evaluation," *Ieee Access*, vol. 8, pp. 153 651–153 670, 2020.